GROUP WORK PROJECT # 1 MScFE 632: Machine Learning in Finance
GROUP NUMBER: 11073

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Kolapo Adedipe	Nigeria	adcolapsmm9@yahoo.com	
Ridwan Rahma Putra	India	ridwan.rputra@gmail.com	
TITUS WAWERU MWANGI	Kenya	wawerutitus41@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).

Team member 1	**Kolapo Adedipe**
Team member 2	**Ridwan Rahma Putra**
Team member 3	**TITUS WAWERU MWANGI**

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.
Note: You may be required to provide proof of your outreach to non-contributing members upon request.

Marketing Handbook

Category 1: Elastic Nets

Step 1: Basics (definition & keywords)

Elastic Net is a **regularized linear model** that combines the L1 penalty of LASSO (which encourages sparsity / variable selection) with the L2 penalty of Ridge (which stabilizes coefficients under multicollinearity). It is typically used when you have many, potentially correlated predictors and want a model that both shrinks coefficients and selects groups of correlated variables.

Formally, for response y and predictors X, Elastic Net solves:

Equations

$$\beta = arg\,min_{\beta}\,(1/2n)\,\|y - X\beta\|_2^2 + \lambda(\alpha\|\beta\|_1 + ((1-\alpha)/2)\|\beta\|_2^2)$$

It is suitable for supervised, penalized linear regression, suitable for prediction, feature selection (sparse solutions), and stabilizing estimates when features are collinear. Works for continuous targets (regression); with generalized-loss forms it can be extended to logistic, Poisson, etc.

Keywords / Tags

- regularization
- L1 + L2 penalty
- sparse + grouped selection
- correlated predictors
- feature selection
- shrinkage
- convex optimization
- hyperparameters: lambda, alpha
- elastic-net path / coordinate descent
- stable out-of-sample performance

Step 2: Elastic Net

Advantages

- Balances LASSO & Ridge: combines variable selection (LASSO) with stability under multicollinearity (Ridge).

- Handles correlated predictors: unlike LASSO, which may arbitrarily drop one, Elastic Net can retain groups of correlated variables.

- Robustness: reduces variance in high dimensional settings.

- Flexibility: tuning parameter α\alphaα allows interpolation between pure LASSO (alpha=1) and pure Ridge (alpha=0).

- Interpretability: produces sparse models (many coefficients exactly zero), making the model easier to explain to investment managers.

Computation

Code [in separate jupyter notebook – Elastic Net code.ipynb]

Disadvantages

- Hyperparameter tuning is essential, performance depends heavily on choice of lambda and alpha.

- Computational cost, cross-validation over a grid of parameters can be expensive for large-scale data.

- Sensitivity to scaling, predictors must be standardized; otherwise, penalties bias variables with larger scales.

- Interpretation challenge, coefficients are shrunk, magnitudes cannot be interpreted as directly as in OLS.

Equations

$$\beta = arg\,min_{\beta}\,(1/2n)\,\|y - X\beta\|_2^{\,2} + \lambda(\alpha\|\beta\|_1 + ((1-\alpha)/2)\|\beta\|_2^{\,2})$$

Features

- Works well in high-dimensional settings.
- Performs automatic variable selection.
- Can retain groups of correlated features.
- Requires data standardization.
- Produces sparse, interpretable models.
- Supports extensions (logistic, multinomial, Poisson regression).

Guide (Inputs / Outputs)

- Inputs:
    - Predictor matrix X (standardized features).
    - Response vector y.
    - Hyperparameters: lambda, alpha.
- Outputs:
    - Estimated coefficients β.
    - Set of selected variables (nonzero coefficients).
    - Predictions y=Xβ.
    - Performance metrics (R square, MSE, cross-validation scores).

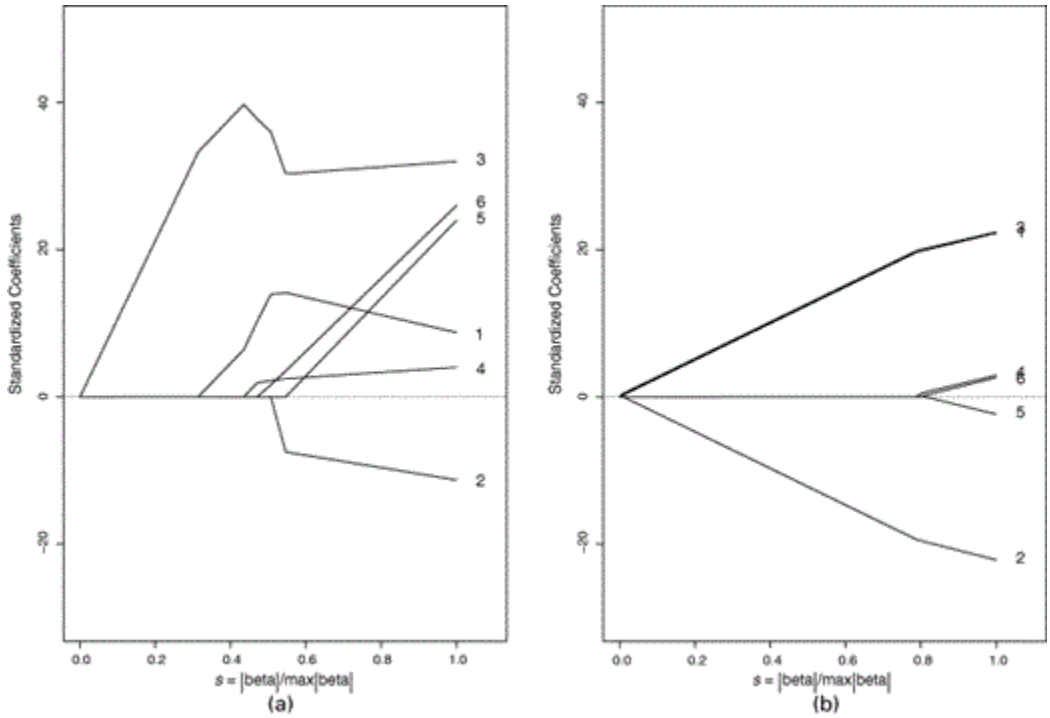
Hyperparameters

- lambda: regularization strength (larger → stronger shrinkage)
- alpha: mixing parameter (0=Ridge, 1=LASSO).
- max_iter: maximum iterations for convergence.
- tol: tolerance for optimization convergence.

- cv folds (if using cross-validation).

Illustration

- **Path plots**: show coefficient shrinkage as lambda increases.



(a) Lasso and (b) elastic net (λ_2=0.5) solution paths: the lasso paths are unstable and (a) does not reveal any correction information by itself; in contrast, the elastic net has much smoother solution paths, while clearly showing the 'grouped selection'—$\mathbf{x}_1$, $\mathbf{x}_2$ and $\mathbf{x}_3$ are in one 'significant' group and $\mathbf{x}_4$, $\mathbf{x}_5$ and $\mathbf{x}_6$ are in the other 'trivial' group; the decorrelation yields the grouping effect and stabilizes the lasso solution (source: hui zou, 2005)

- **Flowchart**:

1. Input data → Standardize
2. Choose grid of alpha, lambda
3. Fit model via coordinate descent
4. Evaluate CV error → select best parameters

5. Output coefficients + predictions

Best alpha : 0.0021209508879201904

Best l1_ratio (α): 1.0

R^2 on test: 0.9999994275069372

Journal

Ho, Michael et al. "Weighted Elastic Net Penalized Mean-Variance Portfolio Design and Computation." *SIAM J. Financial Math.* 6 (2015): 1220-1244.

This paper shows how the elastic net method can be used in finance for building investment portfolios. The usual Markowitz model for portfolio choice often performs poorly because the estimates of returns and risks are not very accurate. To improve this, the authors add a weighted elastic net penalty, which helps by making portfolios simpler (fewer active assets) and more stable against errors in the data.

They also use a bootstrap method (resampling past data) to set the penalty weights in a practical way and create a faster algorithm to solve the problem with many stocks. The study uses U.S. stock data from 2001 to 2014 and finds that the weighted elastic net approach gives better risk-adjusted performance (higher Sharpe ratios) than traditional portfolios and other penalty methods.

In short, the paper shows that elastic net can make portfolio selection more reliable and effective when financial data is uncertain or limited

Category 2: Hierarchical Clustering

STEP 1: Basics (definition & keywords) AND Step 2: Hierarchical Clustering Features

1. In machine learning, hierarchical clustering is an unsupervised algorithm that, without a set number of clusters, arranges data points into a hierarchy of nested clusters.

It uses pairwise distances and a linkage rule (single, complete, average, Ward) to create a tree (dendrogram) of observations. It may be divisive (top-down) or agglomerative

(bottom-up). When merging, Ward's linkage reduces the increase in the overall within-cluster variance.

2. a) Flexible as it does not require the number of clusters to be specified before.

   b) Produces a Dendrogram that visualizes the data and its cluster levels and relationships.

3. a) Agglomerative hierarchical clustering is generally not the best option for very large universes without optimizations due to its $O(N^2)$ memory/time cost.

   b) Linkage selection is important (complete, average, and Ward differ; a single linkage results in chaining).

   c) Single merges are irreversible (no reassignments like k-means); they are less resilient to noisy features.

   d) Suboptimal solutions result from the inability to reverse decisions once a cluster is formed.

4. Unlike K-Means, which is defined by a centroid calculation, hierarchical clustering is not characterized by a single mathematical equation. Rather, it is an algorithm for measuring the distance between clusters that is based on a process and a set of rules.

   The algorithm steps and the distance (or linkage) metrics are the "equations" that form the model.

   a) The Distance metric

      Determines how the similarity or dissimilarity of individual points is calculated.

   The most widely used metric, the Euclidean Distance($d(xi,xj)=\sum k=1p(xik-xjk)2$)), calculates the distance in a straight line between two points in multidimensional space.

   Manhattan Distance (City Block or Taxi): calculates the distance by adding up the absolute differences between their coordinates, simulating a journey along a street grid.

   A generalized form of the Manhattan and Euclidean distances is the Minkowski distance. It is the Manhattan distance when p=1, and the Euclidean distance when p=2.

Cosine similarity is a technique that is frequently applied to high-dimensional data, such as text. It calculates the angle between two vectors to assess how similar they are; the smaller the angle, the more similar the vectors are.

b) Linkage criteria

Based on the distances between each cluster's individual points, the linkage criterion establishes how the distance between two clusters is determined. This one decision significantly impacts the clustering's result.

Let A and B be two clusters. The linkage method defines the distance between them, d(A, B).

Single Linkage (minimum distance between clusters), Complete Linkage (maximum distance), Average Linkage (average distance between all pairs), and Ward's Method (minimizes the increase in error sum of squares) are common linkage techniques in hierarchical clustering.

a) Nearest-Neighbor Single Linkage

The distance between the two points that are closest to each cluster is the distance between two clusters.

The formula is as follows: distance(C1, C2) = min { d(x, y) | x ∈ C1, y ∈ C2 }.

b) Full Connection (Farthest-Neighbor)

Described: The separation between the two furthest points from each cluster is the distance between two clusters.

The formula is as follows: distance(C1, C2) = max { d(x, y) | x ∈ C1, y ∈ C2 }.

c) On average Linkage Description: The average of all pairwise distances between the points in two clusters determines the distance between them.

The formula is as follows: distance(C1, C2) = (1 / |C1|*|C2|) * Σ [d(x, y) | x ∈ C1, y ∈ C2].

d) The two clusters that produce the least increase in the sum of squared errors, or overall within-cluster variance, are combined using Ward's Method.

Formula Idea: reduces Σ((x - mean_C)^2) for every cluster that has been merged.

e) A variant of Average Linkage, Weighted Group Average (WPGMA) allocates weights to the distances according to the cluster sizes.

Description of Centroid Linkage: Determines the separation between the clusters' centroids, or mean points.

5. focuses on engineered features (fundamentals, volatility, momentum) or raw returns.

A taxonomy (dendrogram) produced by hierarchical clustering is reliable for analyzing sector relationships.

K-means is a quick and effective method for creating distinct asset baskets.

Both approaches don't require labels and are unsupervised.

Results are significantly impacted by preprocessing (standardization, handling of missing values).

The distance metric (Euclidean, correlation-based, or Mahalanobis) is important.

6. Inputs
   a) Dataset : raw data typically a 2D array or matrix of shape
   b) Distance matric
   c) Linkage criterion
   d) Number of clusters

Outputs

a) The linkage matrix : A reduced mathematical representation of the dendrogram. The entire history of which clusters were merged, at what distance, and how many original points they contained is recorded in this (n-1) x 4 matrix.
b) The Dendrogram : a tree diagram that shows the linkage matrix's hierarchical merging process.

c) Cluster Labels : For every data point, a flat list of cluster assignments is produced by "cutting" the linkage matrix or dendrogram at a given height or for a predetermined number of clusters.

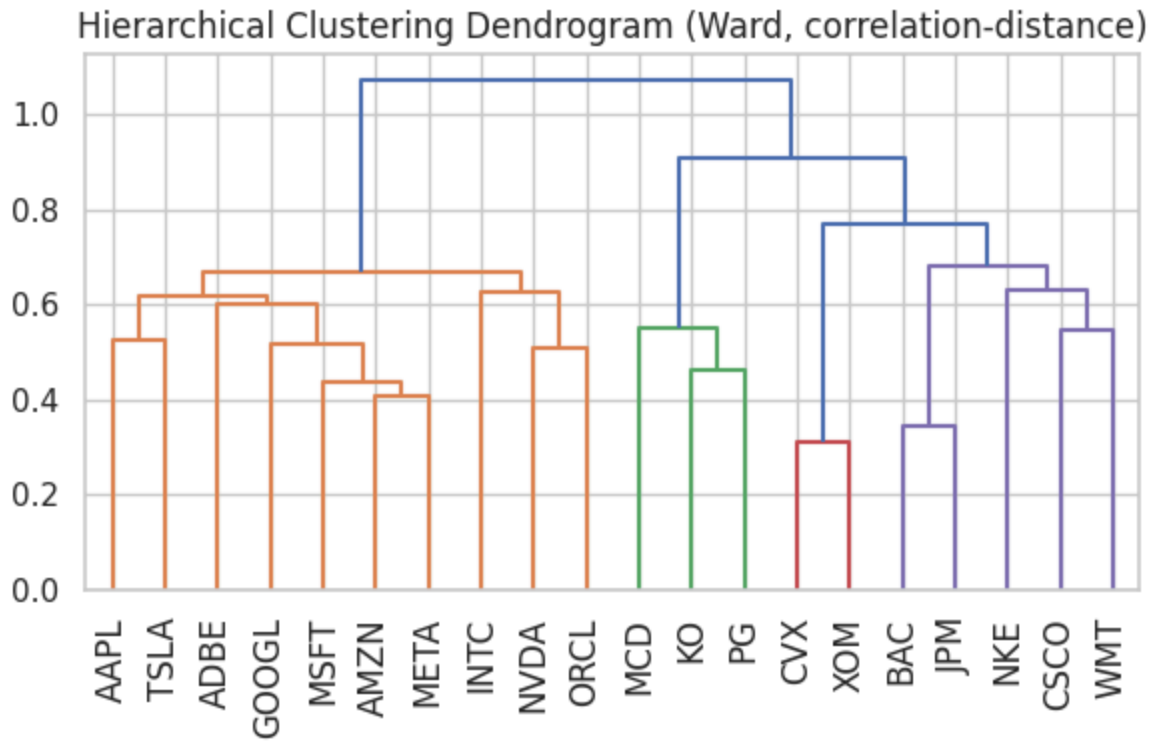
7. There aren't many hyperparameters for Hierarchical Clustering. However, the ideal setting depends greatly on your particular dataset and the objectives of your analysis; there is no "best" setting.

   a) The linkage criterion

   rule that establishes how, following the merging of the initial points, the distance between two clusters is computed. There is no mathematical optimization. It is necessary to test various approaches and assess the resulting clusters and dendrograms using diagnostic metrics (such as cophenetic correlation) or domain knowledge.

   b) Distance metric
      explains the calculation used to determine the distance between two separate data points.

   c) Number of clusters
      This hyperparameter is post-hoc. To obtain a flat list of clusters, the algorithm first constructs the entire hierarchy, after which you can choose where to cut it.



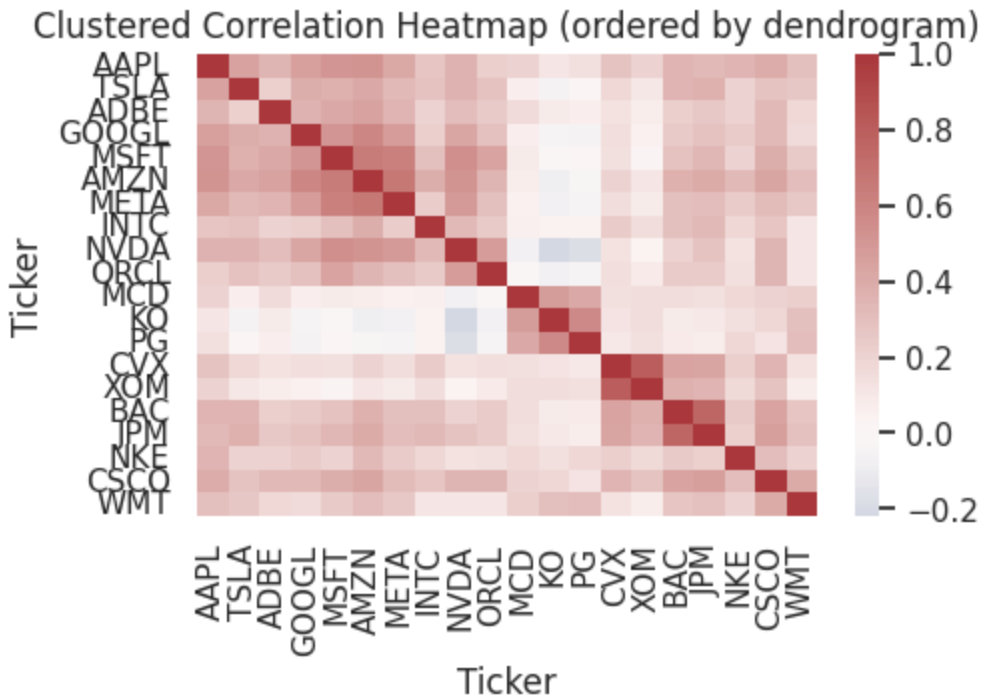
8. Hierarchical dendrogram displaying nested structure.

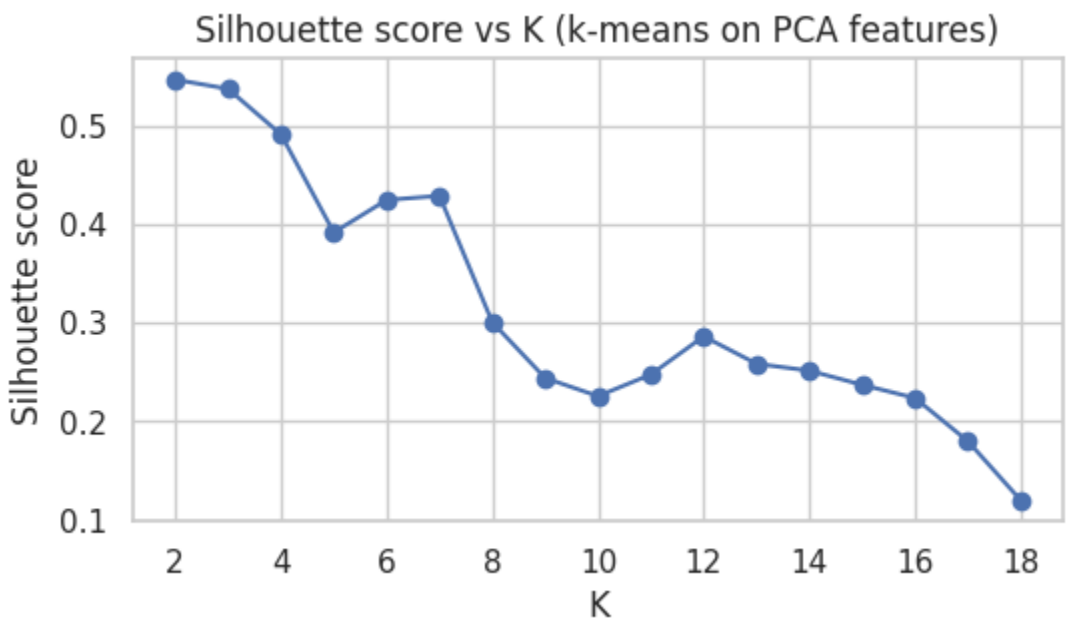
   Heatmap of clustered correlation (correlation matrix arranged according to cluster assignment).

   2D cluster-colored projection (PCA or t-SNE).

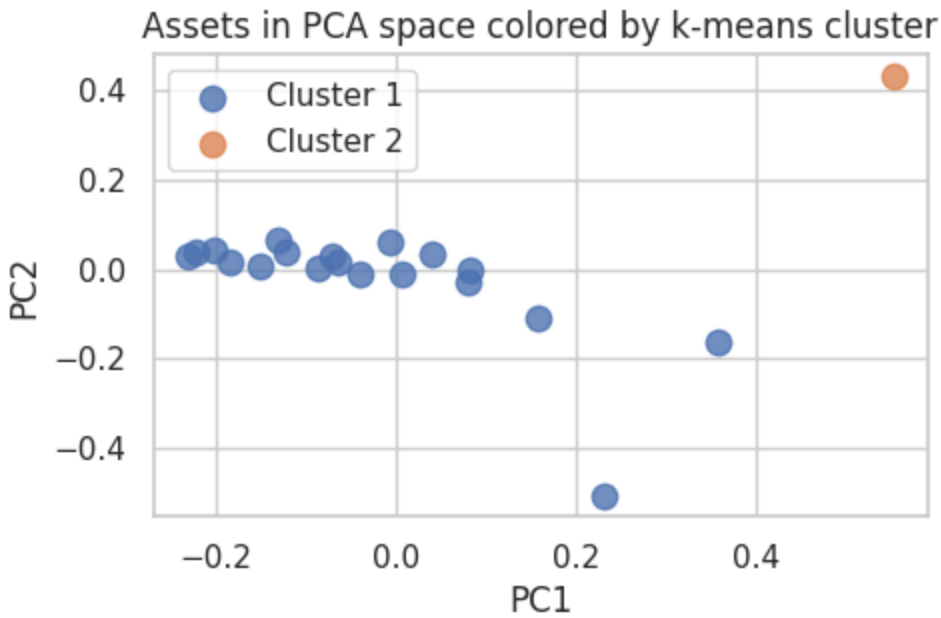
   Bar chart showing the cluster return and volume summary (to evaluate investment qualities).

9. Tumminello, Michele, et al. "Correlation, Hierarchies, and Networks in Financial Markets." Physica A: Statistical Mechanics and its Applications, vol. 389, no. 3, 2010, pp. 2990-94. ScienceDirect, doi:10.1016/j.physa.2010.03.004.





best K by silhouette: 2



10.

Agglomerative Clustering (method from the bottom up)

Divisive Clustering (top-down method)

Clustering based on connectivity

The Ward Method

Full Connection

Average Connection

Clustering based on trees

Merging Step-by-Step

Matrix of Links

Hierarchy of Clusters

Deterministic Outcomes

Visual Outcome (Dendrogram)

No predetermined K (the number of clusters is determined after the fact)

Category 4: Classification Trees

Step 1: Basics and Keywords

Definition

A classification tree is a type of decision tree algorithm used in machine learning and predictive analytics to categorize data into predefined groups or "classes" (**Wikipedia, 2025**).

It works by recursively splitting data into smaller subsets based on decision rules derived from features (independent variables) (**DataCamp, n.d.**).

The end result is a tree-like structure where branches represent decision rules, and leaves represent class labels (outcomes) (**Wikipedia, 2025**).

Classification

- **Supervised learning model**: Requires labeled data (known outcomes) (**Wikipedia, 2025**).

- **Non-linear model**: Unlike linear regression, classification trees can model complex, non-linear relationships (**GeeksforGeeks, n.d.**).

- **Discrete output**: Predictions belong to a category/class (e.g., "yes/no," "churn/not churn," "low/medium/high") (**Wikipedia, 2025**).

Core Concepts

- **Root Node**: The starting point of the tree (**DataCamp, n.d.**).

- **Splits/Branches**: Decision rules (based on variables like age, income, or purchase history) (**DataCamp, n.d.**).

- **Leaf Nodes**: The final decision or classification (**Wikipedia, 2025**).

- **Impurity Measures**: Metrics like Gini Index or Entropy used to decide the best splits (**Wikipedia, 2025**).

Advantages

- Easy to interpret and visualize (**Wikipedia, 2025**).

- Handles both numerical and categorical data (**DataCamp, n.d.**).

- Useful for customer segmentation and marketing campaign targeting (**GeeksforGeeks, n.d.**).

Limitations

- Can overfit (too many branches) (**Analytics Vidhya, 2021**).

- Sensitive to small changes in data (**Analytics Vidhya, 2021**).

- May require ensemble methods (e.g., Random Forest, Gradient Boosting) for better accuracy (**Wikipedia, 2025**).

Keywords for Marketing Context / Tags for Classification Trees

- Customer Segmentation

- Predictive Modeling

- Decision Rules

- Target Audience Classification

- Purchase Behavior Prediction
- Churn Analysis
- Cross-sell & Upsell Targeting
- Data-driven Marketing
- Machine Learning
- Supervised Learning
- Non-linear Models
- Classification Algorithm
- Churn Prediction
- Purchase Behavior Analysis
- Predictive Analytics
- Business Intelligence
- Risk Assessment
- Feature Importance
- Interpretability

(Keywords drawn from typical marketing analytics applications of classification trees: Analytics Vidhya, 2021; DataCamp, n.d.)

Step 2: Classification Trees Features

Advantages of Classification Trees

1. **Interpretability and Simplicity**
   Classification trees are easy to understand and interpret, even for non-technical stakeholders. Their tree-like structure, with decision rules leading to outcomes, makes them transparent and useful for presenting results to decision-makers (**Wikipedia, 2025**).

2. **Handling of Diverse Data Types**
   They can process both numerical and categorical variables without requiring complex data transformations, which makes them highly versatile for real-world datasets (**DataCamp, n.d.**).

3. **Modeling Non-linear Relationships**
   Unlike linear models, classification trees capture complex, non-linear relationships between features and outcomes, which improves their applicability across a wide range of domains (**GeeksforGeeks, n.d.**).

4. **Flexibility in Marketing and Business Applications**
   They are particularly effective in marketing tasks such as customer segmentation, churn prediction, and purchase behavior analysis, where categories (e.g., “churn/not churn”) are more meaningful than continuous predictions (**Analytics Vidhya, 2021**).

5. **No Need for Assumptions About Data Distribution**
   Classification trees do not assume linearity, normality, or homoscedasticity in the data, making them robust when applied to diverse datasets (**Wikipedia, 2025**).

Why This Analysis Matters for Marketers

The **Telco Customer Churn dataset** represents a common business problem: identifying which customers are likely to leave (churn) so that marketing teams can take action.

Using a **Classification Tree** helps marketers because:

- **Customer Segmentation**: The tree splits customers based on key attributes (e.g., contract type, tenure, monthly charges), creating natural segments.
- **Actionable Insights**: Each branch shows *decision rules* that help marketing teams target the right customers with retention offers.

- **Churn Reduction Campaigns**: By predicting churn, companies can design proactive campaigns (discounts, loyalty programs, better service bundles) to retain high-risk customers.
- **Resource Allocation**: Marketing budgets can be focused on customers with the highest probability of leaving, improving ROI.

In short, classification trees transform **raw customer data into clear decision rules**, empowering marketers to take **data-driven actions** that improve customer lifetime value and reduce churn.

How Marketing Teams Can Use This Output for Churn Campaigns

The classification tree analysis not only predicts churn but also highlights the **top factors influencing customer decisions** (e.g., contract type, tenure, monthly charges).

Marketing teams can apply these insights in the following ways:

- **Targeted Retention Offers**: Customers flagged as high-risk can be offered discounts, flexible contracts, or loyalty rewards.
- **Proactive Engagement**: High-value customers nearing the end of a short-term contract can be contacted early with personalized renewal campaigns.
- **Service Improvement**: If features like *tech support* or *internet service type* are strong churn predictors, marketers can collaborate with operations to enhance these services.
- **Budget Optimization**: Instead of broad retention campaigns, resources can be focused on the **most at-risk segments**, maximizing ROI.

In summary, the tree provides **actionable rules** that connect data science with marketing strategy, enabling churn reduction campaigns that are **smarter, more efficient, and customer-focused**.

Disadvantages of Classification Trees

While classification trees are powerful and easy to interpret, they also come with several **limitations**:

1. **Overfitting**

    - Trees can become overly complex, capturing noise in the data rather than true patterns.

- This reduces their ability to generalize to unseen data (Hastie, Tibshirani, & Friedman, 2009).

2. **Instability**

   - Small changes in the dataset (e.g., adding or removing a few records) can lead to a **completely different tree structure**, making results less reliable (Loh, 2014).

3. **Bias Toward Features with Many Levels**

   - Features with more categories (e.g., ZIP codes) are more likely to be chosen for splits, even if they are not the most informative (Breiman et al., 1984).

4. **Limited Predictive Power (Compared to Ensembles)**

   - A single classification tree often performs worse than ensemble methods such as **Random Forests** or **Gradient Boosting**, which reduce variance and improve accuracy (Lantz, 2019).

5. **Difficulty with Linear Relationships**

   - Trees split data into rectangular regions, which makes them less effective at modeling smooth or linear boundaries between classes (Han, Kamber, & Pei, 2011).

Equations: How a Classification Tree Works

1. **Node Impurity**
   At each node, the algorithm selects a split that **reduces impurity**. Common measures:

   Gini Index:

$$Gini(t) = 1 - \sum_{i=1}^{C} p_i^2 \#\#$$

- $p_i \#\#$ = proportion of class i in node t
- C = total number of classes
- Lower Gini → purer node

Entropy (Information Gain):

$$Entropy(t) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

- Measures the disorder in a node.
- Splits are chosen to **maximize information gain**:

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- S = current dataset
- A = feature used for splitting
- S_v = subset of S where feature A has value v

2. **Decision Rule / Prediction at Leaf Node**

After building the tree, a leaf node assigns a class based on **majority voting**:

$$\hat{y} = \arg \max_{i \in \{1,\ldots,C\}} p_i$$

- $\hat{y}$ = predicted class

- p_i = proportion of class iii in the leaf node

3. **Tree Growth (Recursive Partitioning)**

The tree is grown recursively by splitting nodes:

$$t_{left}, t_{right} = \arg\min_{A,v} \left[\frac{|t_{left}|}{|t|} Impurity(t_{left}) + \frac{|t_{right}|}{|t|} Impurity(t_{right}) \right]$$

- A = candidate feature

- v = threshold value for the split

- t_{left}, t_{right} = resulting child nodes

- $Impurity(\cdot)$ = Gini or Entropy

The algorithm stops splitting when:

- Maximum depth is reached, or

- Node is pure ($Gini = 0$), or

- Minimum samples per leaf is reached

Summary

- Classification trees **split data recursively** based on features to minimize impurity.

- Leaves **predict the class** via majority vote.

- Impurity measures (Gini or Entropy) guide the best splits.

Features of Classification Trees

1. **Handles Both Numerical and Categorical Data**
    - Can split on continuous features (e.g., monthly spend) or categorical features (e.g., contract type) without special encoding.
2. **Non-Parametric / No Distribution Assumptions**
    - Does not assume linearity, normality, or homoscedasticity in the data.
3. **Handles Missing Values**
    - Some implementations (e.g., CART, C4.5) can handle missing values either by surrogate splits or by ignoring missing observations at a node.
4. **Interpretable and Visualizable**
    - Produces a tree structure that is easy to understand for non-technical stakeholders.
    - Decision rules from the tree can be expressed in “if-then” statements.
5. **Automatic Feature Selection**
    - Splitting criteria prioritize the most informative features at each node, effectively performing feature selection.
6. **Flexible and Robust**
    - Can model complex, non-linear relationships and interactions between variables.
    - Can be used for classification tasks with multiple classes.

7. **Supports Ensemble Methods**

    - Works as a base learner in ensemble techniques like **Random Forests** and **Gradient Boosting**, which improve accuracy and reduce overfitting.

Optional Note for Marketing Context

- Classification trees are particularly useful in **customer segmentation, churn prediction, and targeted marketing**, because the features selected by the tree naturally highlight the most important factors driving behavior.

Guide: Inputs and Outputs of Classification Trees

Inputs

1. **Training Dataset**

    - Independent variables (**features/predictors**)

        - Can be **numerical** (e.g., income, tenure, monthly charges)

        - Or **categorical** (e.g., gender, contract type, payment method)

    - Dependent variable (**target/class**)

        - Must be **categorical** (e.g., churn = Yes/No; customer type = High/Low value)

2. **Hyperparameters** (model configuration)

    - max_depth: maximum levels of the tree

    - min_samples_split: minimum samples required to split a node

- criterion: measure of impurity (e.g., Gini, Entropy)
- min_samples_leaf: minimum samples per leaf node

3. **Splitting Criteria**
   - Gini Index
   - Entropy / Information Gain

Outputs

1. **Decision Tree Model**
   - A tree structure of **decision rules** (e.g., "If contract = month-to-month and tenure < 12 → High churn risk").
   - Can be visualized for interpretation.
2. **Predictions**
   - For new/unseen data, the model predicts the class label.
   - Example: Churn = Yes or Churn = No.
3. **Class Probabilities**
   - The proportion of samples per class at the leaf node.
   - Example: {Yes: 0.8, No: 0.2} → predicted class is **Yes** with 80% probability.
4. **Performance Metrics**
   - Accuracy, Precision, Recall, F1-score, ROC-AUC (depending on evaluation).

5. **Feature Importance**
    - Ranking of features based on their contribution to splits and overall predictive power.

Hyperparameters of Classification Trees

1. **criterion**
    - Function to measure the quality of a split.
    - Options:
        - "gini" → Gini Impurity
        - "entropy" → Information Gain (Shannon entropy)
        - "log_loss" → Logarithmic loss (scikit-learn 1.1+)
2. **max_depth**
    - Maximum depth of the tree.
    - Controls model complexity and prevents overfitting.
3. **min_samples_split**
    - Minimum number of samples required to split a node.
    - Larger values → simpler tree, less overfitting.
4. **min_samples_leaf**

- Minimum number of samples required at a leaf node.
- Ensures leaf nodes are not too specific.

5. **max_features**
   - Number of features to consider when looking for the best split.
   - Options: integer, float, "sqrt", "log2", or None.
6. **splitter**
   - Strategy used to choose the split at each node.
   - Options: "best" (default) or "random".
7. **max_leaf_nodes**
   - Maximum number of leaf nodes.
   - Prunes the tree by limiting growth.
8. **min_impurity_decrease**
   - A node will be split only if the impurity decrease is greater than this threshold.
9. **class_weight**
   - Assigns weights to classes (useful for imbalanced datasets).
   - Options: None, "balanced", or a dictionary.
10. **ccp_alpha** (Cost-Complexity Pruning)

- Complexity parameter used for **Minimal Cost-Complexity Pruning**.

- Larger values prune more aggressively.

GridSearchCV vs RandomizedSearchCV

When tuning hyperparameters for classification trees (or any machine learning model), two common strategies are:

1. GridSearchCV

- **How it works**: Tests **all possible combinations** of hyperparameters you define.
- **Pros**: Exhaustive and guarantees finding the best parameter set within the grid.
- **Cons**: Computationally expensive (slow) when the grid is large or the dataset is big.
- **Best for**:
  - Small to medium datasets
  - When you want full control over specific parameter values

2. RandomizedSearchCV

- **How it works**: Randomly samples a fixed number (n_iter) of parameter combinations from predefined distributions.
- **Pros**: Much **faster**; covers the search space without needing every combination.
- **Cons**: May miss the exact "best" combination, but usually finds a very good one.
- **Best for**:
  - Large datasets
  - When computation time is limited
  - Early-stage exploration of hyperparameter space

Rule of Thumb for Marketers and Data Teams

- Use **RandomizedSearchCV** first to quickly find a promising parameter region.
- Then refine with **GridSearchCV** on a smaller, focused grid around those values.

This hybrid approach balances **efficiency** with **accuracy** and is especially practical for churn prediction or purchase modeling where datasets can be large.

Illustrations: How Classification Trees Work

1. Conceptual Flowchart

A simple diagram showing the **process**:

- Input data → Splitting based on features → Decision rules → Leaf prediction (class).
  This matches the earlier "splits → impurity → leaf prediction" diagram you requested.

Reference: Han, J., Pei, J., & Kamber, M. (2011). *Data mining: Concepts and techniques* (3rd ed.). Elsevier.

2. Example Classification Tree

Visualize a **real tree** from the Telco Churn dataset:

- Root node splits on **tenure** (e.g., > 12 months vs. ≤ 12 months).
- Branches split further on features like **MonthlyCharges** or **Contract type**.
- Leaves predict **Churn = Yes/No**.

Reference: Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

3. Impurity Reduction Graph

A small plot of **Gini Index / Entropy vs. Node Depth**, showing how impurity decreases as splits occur.

- Useful to visually explain why trees split where they do.

Reference: Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Chapman & Hall/CRC.

4. Feature Importance Bar Chart

Rank the most influential variables for churn prediction:

- E.g., Contract, MonthlyCharges, tenure.
- Helps marketers see **what drives churn**.

Already part of your Jupyter notebook can be exported as a figure.

5. Performance Metrics Visualization

Side-by-side bar plots or confusion matrix showing model performance:

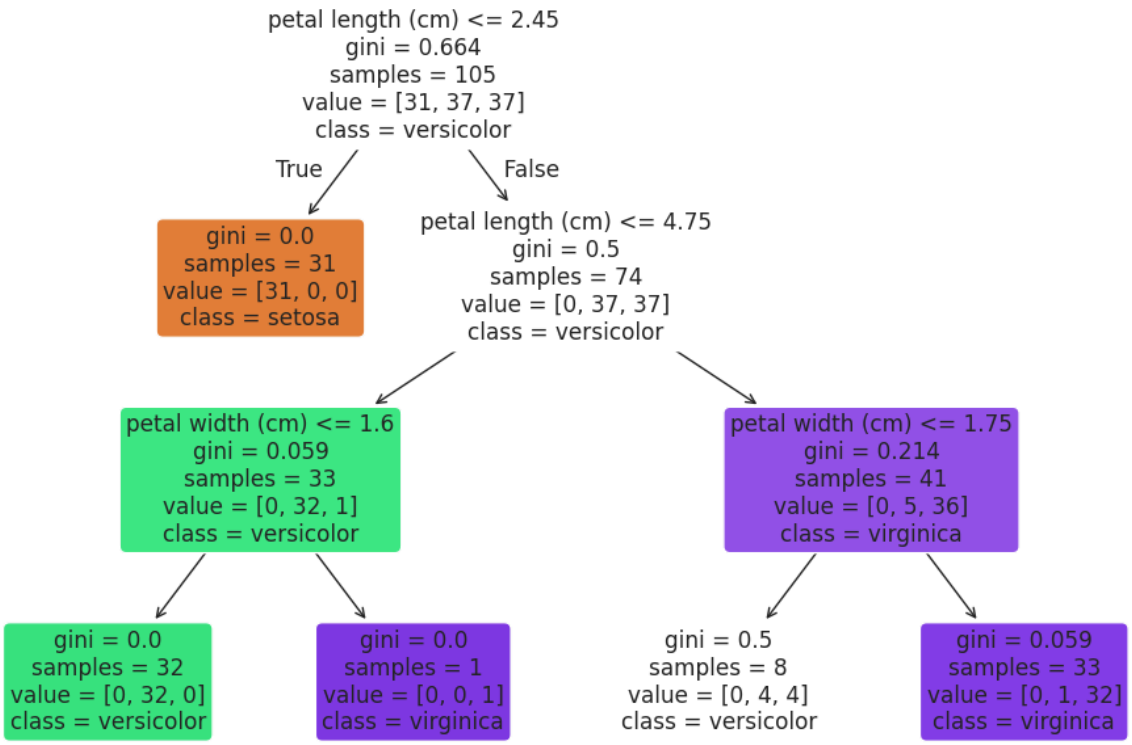
- Accuracy, Precision, Recall, F1.
- ROC curve (Yes vs No churn).

This connects the **mathematics → visuals → marketing application**.

Reference: Provost, F., & Fawcett, T. (2013). *Data science for business: What you need to know about data mining and data-analytic thinking*. O'Reilly Media.

Journal Article Reference:

Ocal, N., Ercan, M. K., & Kadioglu, E. (2016). *Predicting Financial Failure Using Decision Tree Algorithms: An Empirical Test on the Manufacturing Industry at Borsa Istanbul. International Journal of Economics and Finance*, **7**(7), 189-199. CCSE



Accuracy: 1.0

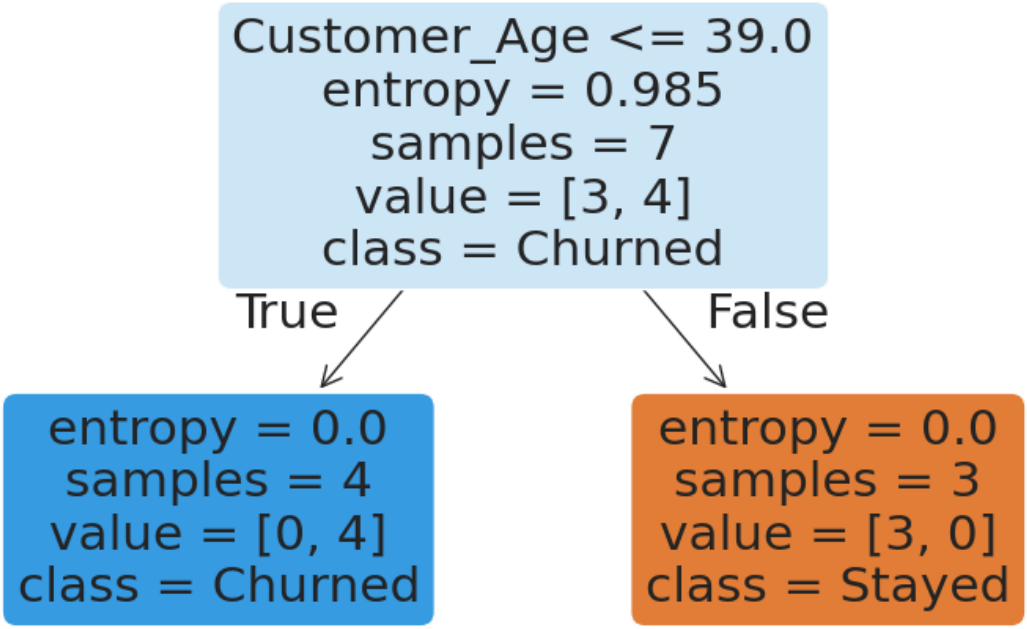
Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13

accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Sample Data:

	Customer_Age	Monthly_Spend	Tenure_Months	Support_Calls	Churn
0	25	40	3	1	1
1	45	90	24	4	0
2	33	65	12	2	1
3	50	120	36	5	0
4	29	50	6	1	1



Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
Stayed	1.00	1.00	1.00	2
Churned	1.00	1.00	1.00	1
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3

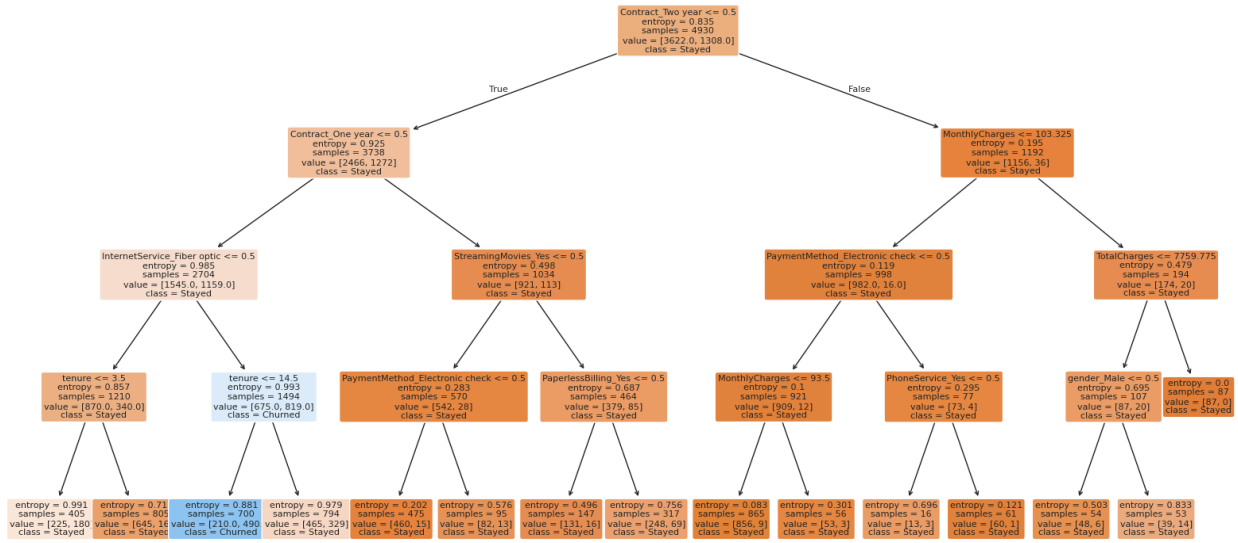
```
weighted avg       1.00      1.00      1.00        3
```

```
Using Colab cache for faster access to the 'telco-customer-churn' dataset.

Downloaded dataset via KaggleHub.

Dataset shape: (7043, 21)

   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \

0  7590-VHVEG  Female              0     Yes        No       1           No

1  5575-GNVDE    Male              0      No        No      34          Yes

2  3668-QPYBK    Male              0      No        No       2          Yes

3  7795-CFOCW    Male              0      No        No      45           No

4  9237-HQITU  Female              0      No        No       2          Yes


     MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \

0  No phone service           DSL            No  ...               No

1               No           DSL           Yes  ...              Yes

2               No           DSL           Yes  ...               No

3  No phone service           DSL           Yes  ...              Yes

4               No     Fiber optic           No  ...               No


  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
```

0	No	No	No	Month-to-month	Yes
1	No	No	No	One year	No
2	No	No	No	Month-to-month	Yes
3	Yes	No	No	One year	No
4	No	No	No	Month-to-month	Yes

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

[5 rows x 21 columns]



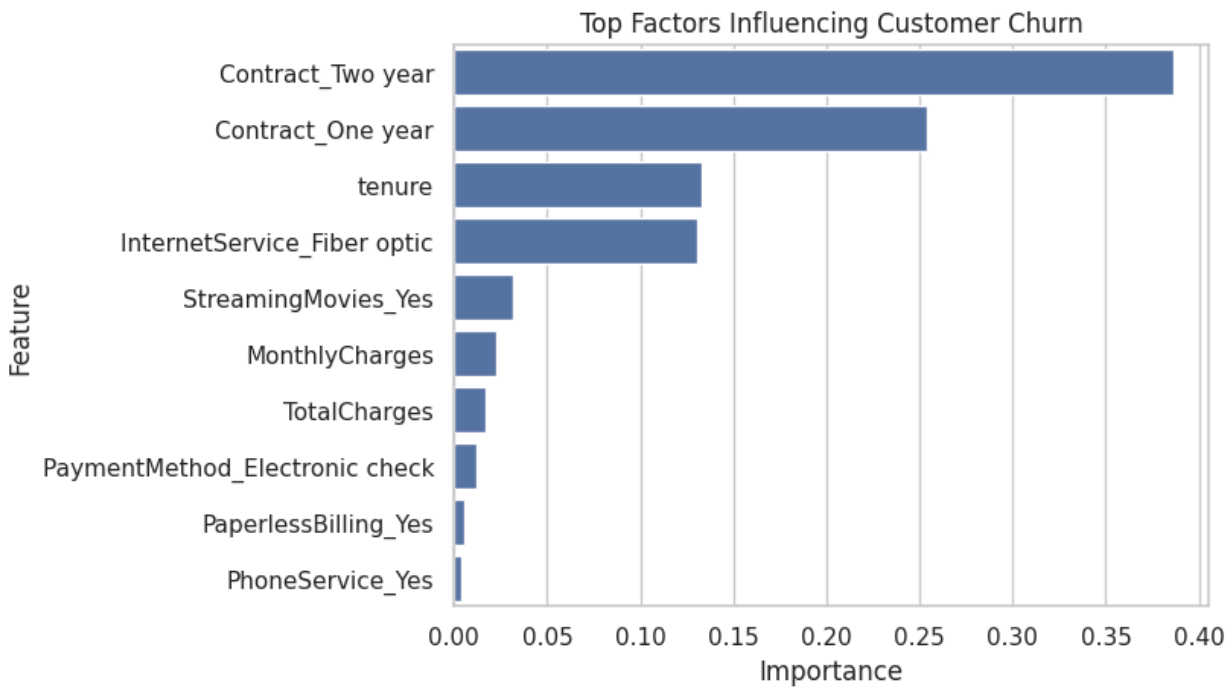
Accuracy on test set: 0.7875059157595835

Classification Report:

	precision	recall	f1-score	support
Stayed	0.80	0.94	0.87	1552
Churned	0.69	0.36	0.47	561
accuracy			0.79	2113
macro avg	0.75	0.65	0.67	2113
weighted avg	0.77	0.79	0.76	2113

Top 10 Features Influencing Churn:

	Feature	Importance
25	Contract_Two year	0.386092
24	Contract_One year	0.254277
1	tenure	0.133098
10	InternetService_Fiber optic	0.130609
23	StreamingMovies_Yes	0.031684
2	MonthlyCharges	0.022779
3	TotalCharges	0.016857
28	PaymentMethod_Electronic check	0.012394
26	PaperlessBilling_Yes	0.005630
7	PhoneService_Yes	0.003811



Fitting 5 folds for each of 648 candidates, totalling 3240 fits

Best Parameters: {'class_weight': None, 'criterion': 'gini', 'max_depth': 5, 'max_features': None, 'min_samples_leaf': 5, 'min_samples_split': 2}

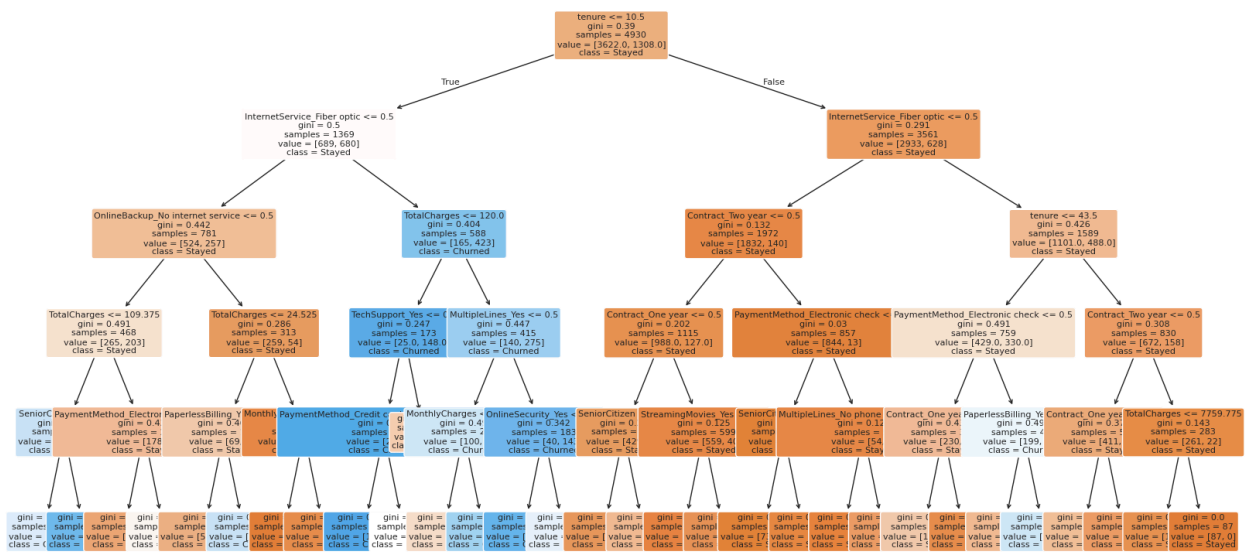
Best CV Accuracy: 0.7892494929006085

Test Set Accuracy: 0.7879791765262659

Classification Report (Tuned Model):

	precision	recall	f1-score	support
Stayed	**0.84**	**0.89**	**0.86**	**1552**

Churned	**0.62**	**0.52**	**0.56**	**561**
accuracy			**0.79**	**2113**
macro avg	**0.73**	**0.70**	**0.71**	**2113**
weighted avg	**0.78**	**0.79**	**0.78**	**2113**



Top 10 Features Influencing Churn (Tuned Model):

	Feature	Importance
1	**tenure**	**0.403541**
10	**InternetService_Fiber optic**	**0.322926**
3	**TotalCharges**	**0.061136**

14 OnlineBackup_No internet service 0.041266

28 PaymentMethod_Electronic check 0.037132

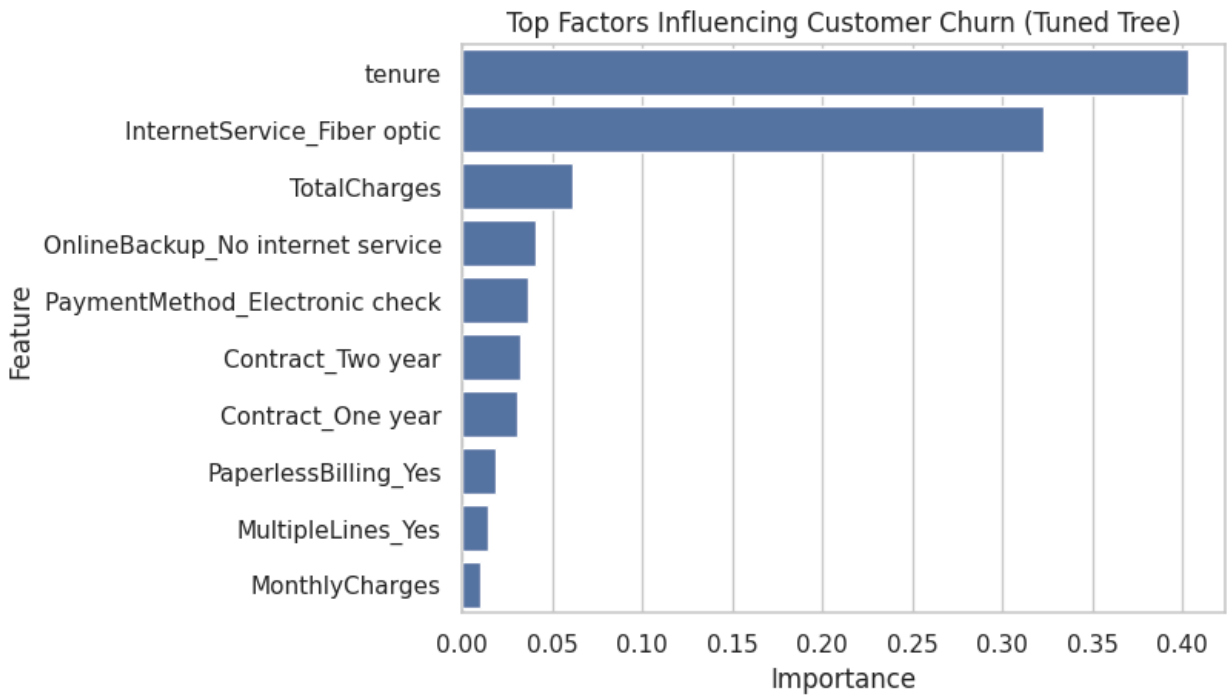
25 Contract_Two year 0.032788

24 Contract_One year 0.031053

26 PaperlessBilling_Yes 0.018688

9 MultipleLines_Yes 0.014887

2 MonthlyCharges 0.010076



Fitting 5 folds for each of 648 candidates, totalling 3240 fits

Best Parameters (GridSearchCV): {'class_weight': None, 'criterion': 'gini', 'max_depth': 5, 'max_features': None, 'min_samples_leaf': 5, 'min_samples_split': 2}

Best CV Accuracy (GridSearchCV): 0.7892494929006085

Fitting 5 folds for each of 50 candidates, totalling 250 fits

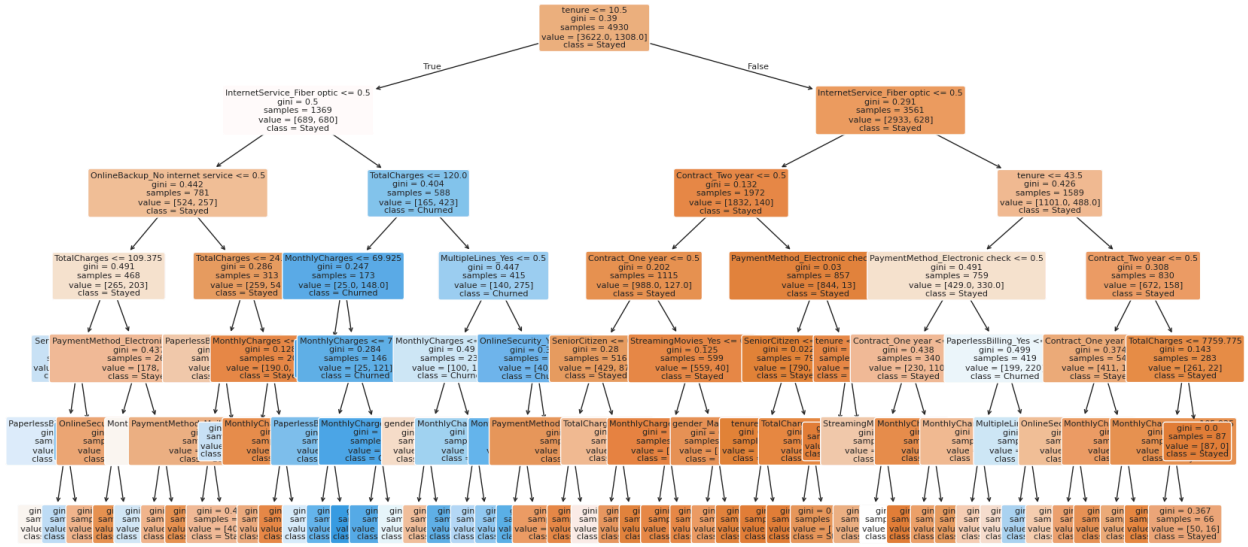
Best Parameters (RandomizedSearchCV): {'class_weight': None, 'criterion': 'gini', 'max_depth': 6, 'max_features': None, 'min_samples_leaf': 17, 'min_samples_split': 18}

Best CV Accuracy (RandomizedSearchCV): 0.7937119675456389

Test Set Accuracy: 0.7917652626597255

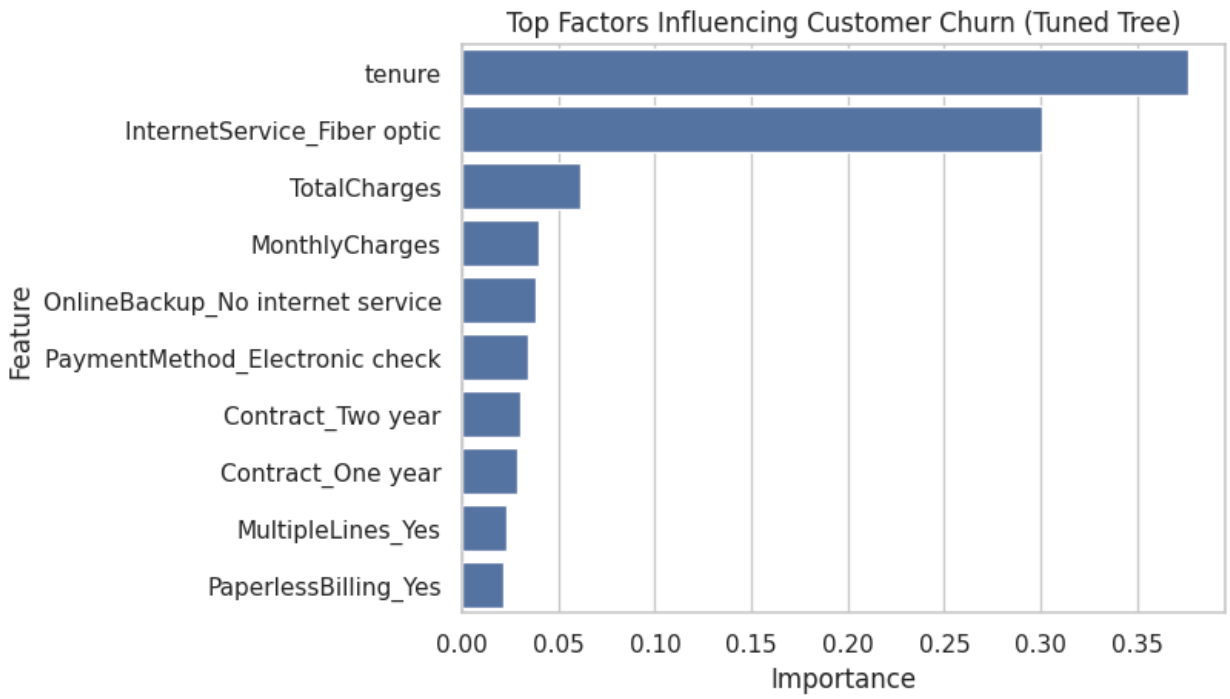
Classification Report (Tuned Model):

	precision	recall	f1-score	support
Stayed	0.83	0.90	0.86	1552
Churned	0.64	0.49	0.55	561
accuracy			0.79	2113
macro avg	0.74	0.69	0.71	2113
weighted avg	0.78	0.79	0.78	2113



Top 10 Features Influencing Churn (Tuned Model):

	Feature	Importance
1	tenure	0.376725
10	InternetService_Fiber optic	0.300809
3	TotalCharges	0.061790
2	MonthlyCharges	0.039537
14	OnlineBackup_No internet service	0.038440
28	PaymentMethod_Electronic check	0.034589
25	Contract_Two year	0.030543
24	Contract_One year	0.028926
9	MultipleLines_Yes	0.023331
26	PaperlessBilling_Yes	0.021759



Step 3: Technical Section

Introduction

Machine learning algorithms' hyperparameters are modifications made prior to training that have direct control over the learning process. In contrast to model parameters (like regression coefficients or decision tree splits), hyperparameters need to be specified by the researcher and are not learned from the data. The model's performance, resilience, and capacity for generalization can all be greatly affected by their values.

These external configuration variables are established just before the start of the training procedure. They serve as the "knobs" and "dials" that regulate the very essence of the learning process itself rather than being learned from the data. To maximize a model's performance and generalizability to new, unseen data, the right hyperparameters must be chosen. Hyperparameter tuning or hyperparameter optimization are terms used to describe this procedure.Overfitting (model too complex and not generalizable) or underfitting (model too simple) can result from poorly adjusted hyperparameters.

Therefore, fine-tuning hyperparameters is an essential step in creating trustworthy trading strategies, particularly in the financial industry where data can be noisy, non-stationary, and prone to overfitting.

Approaches to hyperparameter tuning

i) Grid Search

tests every possible combination of hyperparameter values within specified ranges in an exhaustive manner.

The advantage: Easy to use and ensures that all values are tested.

Costly to compute, particularly for high-dimensional hyperparameter spaces, is a drawback.

ii) Random search

randomly selects hyperparameter combinations from predetermined distributions or ranges.

Because not all hyperparameters have an equal impact on performance, it is frequently more effective than grid search.

iii) Cross-validation(CV)

used to evaluate model performance across several data folds in conjunction with grid or random search.

lowers the possibility of overfitting to a single split of tests and trains.

iv) Bayesian optimization

models the function of mapping hyperparameters to performance using probabilistic models (such as Gaussian processes). chooses the most promising hyperparameters iteratively.

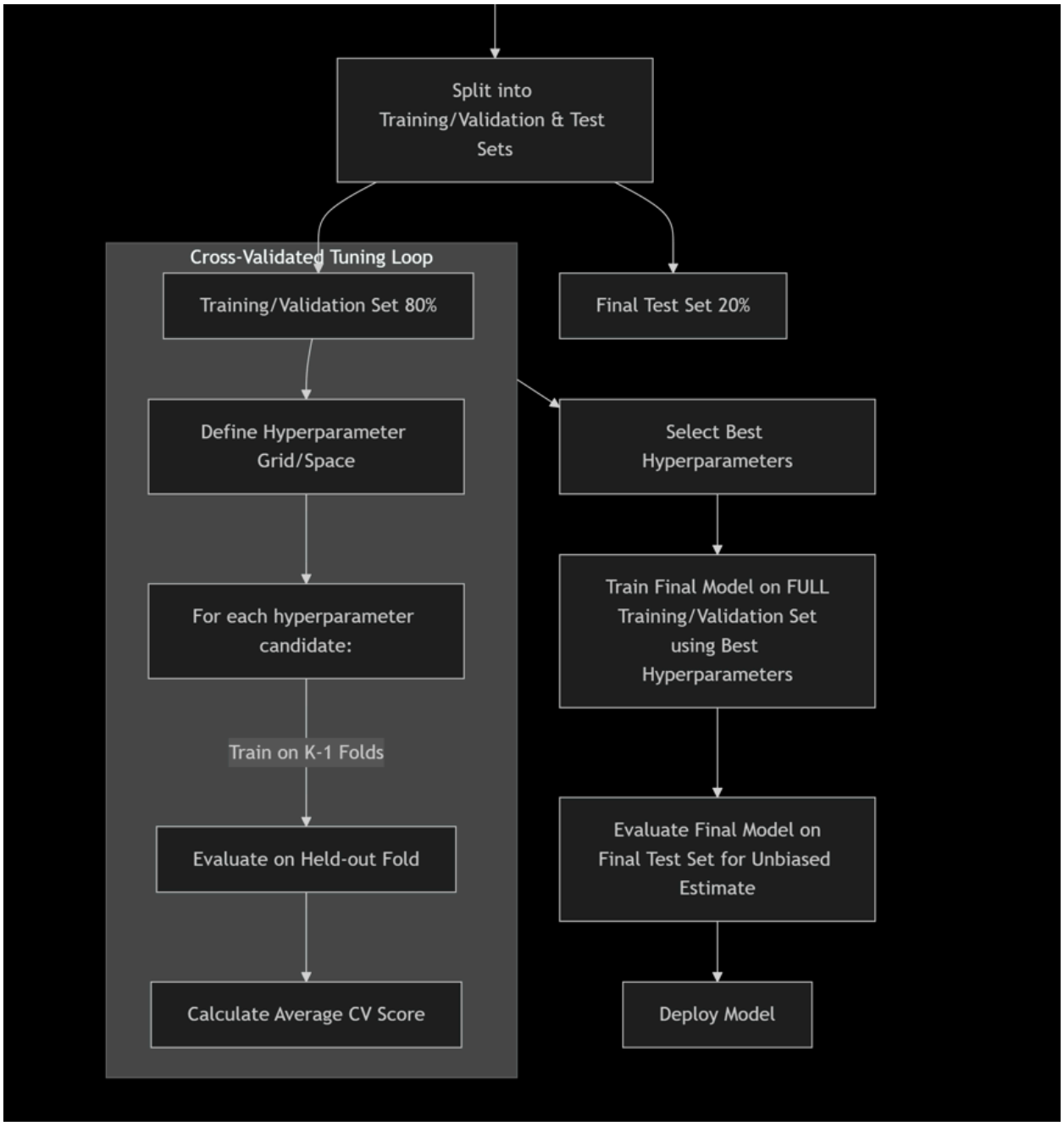
more effective for pricey models than grid or random search.

v) Gradient based optimization

Useful in situations where the hyperparameter space is differentiable, such as neural net learning rates.

used more frequently in deep learning but less frequently in simpler econometric-style models.

The Tuning workflow


Split into Training/Validation & Test Sets
Cross-Validated Tuning Loop
Training/Validation Set 80%
Final Test Set 20%
Define Hyperparameter Grid/Space
Select Best Hyperparameters
For each hyperparameter candidate:
Train Final Model on FULL Training/Validation Set using Best Hyperparameters
Train on K-1 Folds
Evaluate on Held-out Fold
Evaluate Final Model on Final Test Set for Unbiased Estimate
Calculate Average CV Score
Deploy Model

Examples from our models

1. **Category 2**

   i) Lasso Regression(L1 regularization)

   The regularization strength, alpha (often represented by λ), is a crucial hyperparameter. Need for Tuning: Little penalty is added by a small alpha, which increases the chance of dense coefficients and overfitting. A high alpha runs the risk of underfitting and producing an unduly sparse model by aggressively shrinking coefficients towards zero. Accurate prediction and feature selection depend on the ideal value.

   ii) Ridge Regression(L2 regularization)

   Much like with LASSO, underfitting results from a large alpha, while overfitting results from a small alpha. Ridge, on the other hand, reduces coefficients proportionately without making them zero, so tuning only considers the trade-off between bias and variance without feature selection.

   iii) Elastic Net(L1 + L2 Regularization)

   Important hyperparameters: alpha (also known as λ): The strength of regularization overall.
   The mix ratio between the L1 and L2 penalties is known as the l1_ratio (or α).
   Tuning Need: A two-dimensional search is necessary for this model. To find the best combination of Ridge's group effect and LASSO's sparsity, the l1_ratio needs to be adjusted, particularly for correlated financial features. Pure LASSO has a ratio of 1, and pure Ridge has a ratio of 0.

2. **Category 2**

   i) K-means Clustering

   The number of clusters to be found is the key hyperparameter, n_clusters (k).

The most basic decision is the tuning needed. A value that is too high results in overfitting and arbitrary, meaningless partitions, while a value that is too low misses the underlying structure of the data. It is adjusted by assessing metrics such as the elbow method on within-cluster sum of squares, the Davies-Bouldin index, and the silhouette score.

ii) Hierarchical clustering

Important Hyperparameters: n_clusters: The number of clusters (if the dendrogram is being cut at a particular level).
The height at which to cut the dendrogram is known as the distance_threshold.
Linkage: The technique (e.g., 'ward', 'complete', 'average') used to calculate the distance between clusters.
Need for Tuning: In contrast to k-means, the number of clusters is based on where you cut the tree and is not preset. The dendrogram's structure and the resulting clusters are significantly altered by the linkage hyperparameter selection, which needs to be adjusted according to the desired cluster properties (e.g., compactness, variance).

3. **Category 4**

i) Classification Trees

Important hyperparameters include max_depth (the tree's maximum depth), min_samples_split (the minimum number of samples needed to split a node), min_samples_leaf (the minimum number of samples needed at a leaf node), and ccp_alpha (the pruning complexity parameter).
Tuning Need: Unless specifically limited, a tree will overfit (grow to fit the training data perfectly). While adjusting ccp_alpha prunes the tree back after it has grown, adjusting max_depth or the min_samples_* parameters prevent growth. The objective is to accurately classify new financial data by striking the correct balance between generalizability and complexity.

ii) Regression Trees

Key Hyperparameters: Max_depth, min_samples_split, min_samples_leaf, and ccp_alpha are the same as for Classification Trees.

Tuning Need: To avoid overfitting, the same idea applies as with classification trees. However, rather than accuracy, regression metrics like Mean Squared Error (MSE) are used to assess the model's performance. In order to predict a continuous target variable (such as future asset returns) as accurately as possible without learning noise, the tuning process optimizes the tree's structure.

Conclusion

A crucial stage in the machine learning process that turns a generic algorithm into a potent, customized predictor is hyperparameter tuning. To create trading strategies that are not only profitable on historical data but also stable and resilient in real-time markets, quants must become proficient in techniques such as cross-validated random search. A practical decision that needs to be made on a project-by-project basis, the tuning method selection (grid, random, or Bayesian) represents a trade-off between computational resources and the exhaustiveness of the search.

Step 4: Marketing Alpha

In modern marketing, **data is the new alpha**. Just as traders seek strategies that consistently outperform the market, marketers seek tools that **anticipate customer behavior, reduce churn, and maximize ROI**. Machine learning (ML) offers this “Marketing Alpha” by transforming raw customer data into actionable strategies.

Our group’s analysis highlights how different ML methods — from **regularization models** to **clustering and classification trees** — can be combined to give marketing teams an edge.

1. Understanding Customers: Segmentation through Clustering

- **K-Means and Hierarchical Clustering** group customers into natural clusters based on similarities in demographics, purchase behavior, or usage patterns.

- This unsupervised approach provides **flexible segmentation** without requiring predefined labels.

- In marketing, clustering translates into:

  - Identifying **high-value vs. at-risk customer groups**.
  - Designing **personalized offers** for specific segments (e.g., premium bundles for loyal users, retention incentives for price-sensitive customers).

- **Visual outputs** such as dendrograms and heatmaps make complex customer relationships easy to interpret for managers.

Alpha Effect: Marketing spend is optimized by targeting the right group at the right time, instead of applying one-size-fits-all campaigns.

2. Predicting Behavior: Classification and Regression Trees

- **Classification Trees** break down churn prediction into **clear decision rules**:
  - "If contract = month-to-month and tenure < 12 months → High churn risk."
- **Advantages:** They handle both numerical and categorical data, require no distributional assumptions, and are easy to visualize.
- For marketers, this means:
  - **Churn reduction campaigns** informed by top drivers (e.g., contract type, monthly charges).
  - **Targeted cross-sell/upsell strategies**, since trees highlight customers most likely to purchase additional services.
- **Regression Trees** extend the idea to predict continuous outcomes such as **lifetime value (LTV)** or **expected monthly spend**.

Alpha Effect: Trees connect customer insights directly to campaign rules, bridging analytics with frontline action.

3. Sharpening the Model: Regularization and Tuning

- **LASSO (L1), Ridge (L2), and Elastic Net** regression models manage the complexity of high-dimensional marketing data (e.g., dozens of demographic and behavioral features).

- They help avoid **overfitting** by penalizing irrelevant features, automatically selecting the most predictive signals.

- **Hyperparameter Tuning** (via GridSearchCV or RandomizedSearchCV) ensures models remain **generalizable** to new customers.

- This is especially important in marketing where datasets are **dynamic** (customer behavior changes with promotions, competitors, or seasons).

Alpha Effect: Regularization reduces noise and pinpoints the **true drivers of churn, loyalty, or conversion**, making campaigns evidence-based rather than intuition-driven.

4. Why This Matters for Marketers

- **Customer Segmentation:** Clustering creates natural segments for personalized campaigns.

- **Predictive Targeting:** Classification trees flag customers most likely to churn or convert.

- **Value Forecasting:** Regression trees predict LTV, guiding long-term retention strategies.

- **Robustness:** Regularization + tuning ensure models don't just explain the past but also forecast the future.

Together, these methods form a **marketing toolkit** where:

- Clustering = *"Who are my customers?"*

- Classification Trees = *"Who will churn?"*
- Regression Trees = *"What will they spend?"*
- Regularization + Tuning = *"Which signals matter most, and how do I keep models reliable?"*

5. Conclusion: The Marketing Alpha Advantage

By combining clustering, trees, and regularization with systematic hyperparameter tuning, marketers gain the equivalent of **alpha in finance**:

- **Smarter targeting** → higher campaign ROI.
- **Reduced churn** → greater customer lifetime value.
- **Actionable insights** → transparency for executives and teams.

Just as financial analysts rely on quantitative models to identify mispriced assets, marketers can use ML to uncover **hidden patterns in customer data**. The result is **competitive advantage** — campaigns that are not only data-driven but strategically optimized for long-term growth.

Step 5: Learn More

For readers who want to explore further, the following references highlight the application of our chosen models in finance and their broader strengths in machine learning.

Elastic Net

Ho, Michael et al. "Weighted Elastic Net Penalized Mean-Variance Portfolio Design and Computation." *SIAM J. Financial Math.* 6 (2015): 1220-1244.

This article demonstrates how Elastic Net can handle many correlated predictors in forecasting, making it highly useful in finance.

Hierarchical Clustering

Tumminello, Michele, et al. "Correlation, Hierarchies, and Networks in Financial Markets." Physica A: Statistical Mechanics and its Applications, vol. 389, no. 3, 2010, pp. 2990-94. ScienceDirect, doi:10.1016/j.physa.2010.03.004.

Classification tree

Ocal, N., Ercan, M. K., & Kadioglu, E. (2016). *Predicting Financial Failure Using Decision Tree Algorithms: An Empirical Test on the Manufacturing Industry at Borsa Istanbul. International Journal of Economics and Finance*, **7**(7), 189-199. CCSE

General ML Strengths

- "Machine Learning in Asset Management." CFA Institute Research Foundation, 2020, https://www.cfainstitute.org/research/foundation/2020/machine-learning-in-asset-management.
  A trusted source on how ML is transforming asset management, emphasizing practical strengths rather than coding tutorials.

- Jordan, Michael I., and Tom M. Mitchell. *"Machine Learning: Trends, Perspectives, and Prospects."* Science, vol. 349, no. 6245, 2015, pp. 255–260.
  Provides a high-level overview of ML's power and impact across disciplines, including finance.

FOOTNOTE

Advantage for using AI: Needing less time to collect information and to evaluate it for accuracy as compared to using bibliographical research.

REFERENCES

Analytics Vidhya. (2021, August). *Understanding the decision tree algorithm*. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2021/08/decision-tree-algorithm/

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Belmont, CA: Wadsworth International Group.

Brian S. Everitt et al. Cluster Analysis, Fifth Edition, Wiley, 2011.

DataCamp. (n.d.). *Decision tree classification tutorial using scikit-learn*. DataCamp. https://www.datacamp.com/tutorial/decision-tree-classification-python

Data Clustering: 50 Years Beyond K-Means, by Anil K. Jain. Letters on Pattern Recognition, vol. 31, no. 8, 2010, pp. 651–666. doi:10.1016/j.patrec.2009.09.011, Elsevier.

GeeksforGeeks. (n.d.). *Decision tree in machine learning*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/decision-tree/

Han, J., Kamber, M., & Pei, J. (2011). *Data mining: Concepts and techniques* (3rd ed.). Elsevier.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

"Hierarchical Structure in Financial Markets" by Rosario N. Mantegna. Condensed Matter and Complex Systems, European Physical Journal B, vol. 11, no. 1, 1999, pp. 193–197. doi:10.1007/s100510050929, Springer.

Lantz, B. (2019). *Machine learning with R: Expert techniques for predictive modeling* (3rd ed.). Packt Publishing Ltd.

Loh, W. Y. (2014). Fifty years of classification and regression trees. *International Statistical Review, 82*(3), 329–348. https://doi.org/10.1111/insr.12016

Manchek A. Wong and John A. Hartigan, "Algorithm AS 136: A K-Means Clustering Algorithm." Royal Statistical Society Journal, Series C (Applied Statistics), vol. 28, no. 1, 1979, pp. 100–108. doi:10.2307/2346830, JSTOR.

Michele Tumminello, Rosario N. Mantegna, and Fabrizio Lillo. "Networks, Hierarchies, and Correlation in Financial Markets." Physica A: Applications of Statistical Mechanics, vol. 389, no. 1, 2010, pp. 61–72. doi:10.1016/j.physa.2009.09.058 Elsevier.

Wikipedia contributors. (2025, September 14). *Decision tree learning*. In *Wikipedia*. https://en.wikipedia.org/wiki/Decision_tree_learning

Wikipedia contributors. (2025, September 14). *Random forest*. In *Wikipedia*. https://en.wikipedia.org/wiki/Random_forest

Wikipedia contributors. (2025, September 14). *Decision tree learning*. In *Wikipedia*. https://en.wikipedia.org/wiki/Decision_tree_learning

Citation

Ho, Michael et al. “Weighted Elastic Net Penalized Mean-Variance Portfolio Design and Computation.” *SIAM J. Financial Math.* 6 (2015): 1220-1244

Hui Zou, Trevor Hastie, Regularization and Variable Selection Via the Elastic Net, *Journal of the Royal Statistical Society Series B: Statistical Methodology*, Volume 67, Issue 2, April 2005, Pages 301–320, https://doi.org/10.1111/j.1467-9868.2005.00503.x